Iteration H2: Connecting to a Slow Payment Processor with Active Job

The code inside the controllers is relatively fast and returns a response to the user quickly. This means we can reliably give users feedback by checking and validating their orders and the users won't have to wait too long for a response.

The more we add to the controller, the slower it'll become. Slow controllers create several problems. First, the user must wait a long time for a response even though the processing that's going on might not be relevant to the user experience. In the previous section, we set up sending email. The user certainly needs to get that email but doesn't need to wait for Rails to format and send it just to show a confirmation in the browser.

The second problem caused by slow code is *timeouts*. A timeout is when Rails, a web server, or a browser decides that a request has taken too long and terminates it. This is jarring to the user *and* to the code because it means the code is interrupted at a potentially odd time. What if we've recorded the order but haven't sent the email? The customer won't get a notification.

In the common case of sending email, Rails handles sending it in the background. We use `deliver_later` to trigger sending an email, and Rails executes that code in the background. This means that users don't have to wait for email to be sent before we render a response. This is a great hidden benefit to Rails' integrated approach to building a web app.

Rails achieves this using Active Job, which is a generic framework for running code in the background. We'll use this framework to connect to the slow payment processor.

To make this change, you'll implement the integration with the payment processor as a method inside `Order`, then have the controller use Active Job to execute that method in a background job. Because the end result will be somewhat complex, you'll write a system test to ensure everything is working